Experiment No. 6: Vacuum Cleaner Problem

AIM

To write a Python program to implement a simple reflex vacuum-cleaner agent that senses its location and the status of the room and cleans a two-room environment (rooms A and B).

ALGORITHM

1. Represent the environment by two rooms, A and B, each of which is either Clean or Dirty. The agent starts in room A. A percept is the pair (location, status).
2. Define the agent function with condition-action rules: if the status is Dirty, the action is Suck; otherwise, if the location is A, move Right; if the location is B, move Left.
3. Sense the percept of the current room and choose the action using the agent function.
4. Perform the action: Suck changes the current room to Clean; Right / Left changes the location of the agent. Count every action as one unit of cost.
5. Print the step number, location, percept, action and the state of both rooms.
6. Repeat steps 3 to 5 until both rooms are Clean; print the total number of actions taken.

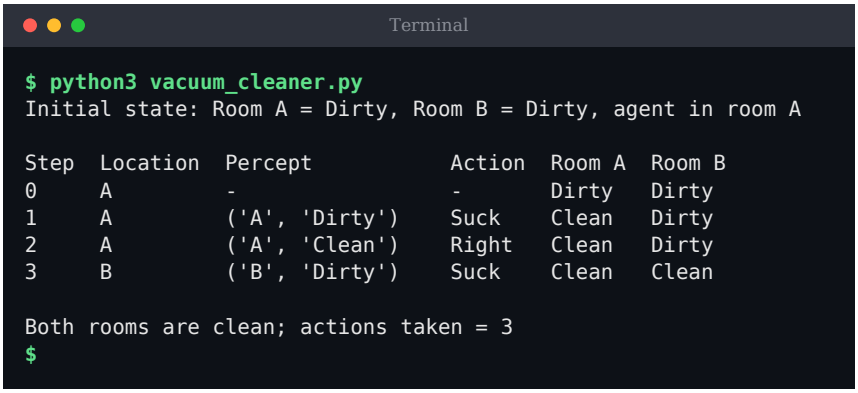
PROGRAM

```
# Vacuum Cleaner problem - simple reflex agent in a two-room world
def reflex_vacuum_agent(percept):
    """percept = (location, status); returns the action chosen by the agent."""
    location, status = percept
    if status == "Dirty":
        return "Suck"
    return "Right" if location == "A" else "Left"

def run(world, location, steps):
    cost = 0                                    # performance measure: number of actions
    print(f"{'Step':<6}{'Location':<10}{'Percept':<18}{'Action':<8}{'Room A':<8}Room B")
    print(f"{0:<6}{location:<10}{'-':<18}{'-':<8}{world['A']:<8}{world['B']}")
    for step in range(1, steps + 1):
        percept = (location, world[location])
        action = reflex_vacuum_agent(percept)
        if action == "Suck":
            world[location] = "Clean"
        elif action == "Right":
            location = "B"
        else:
            location = "A"
        cost += 1
        print(f"{step:<6}{percept[0]:<10}{str(percept):<18}{action:<8}{world['A']:<8}{world['B']}")
        if world["A"] == "Clean" and world["B"] == "Clean":
            break
    return cost

world = {"A": "Dirty", "B": "Dirty"}
print("Initial state: Room A = Dirty, Room B = Dirty, agent in room A\n")
total = run(world, "A", 10)
print(f"\nBoth rooms are clean; actions taken = {total}")
```

OUTPUT



RESULT

Thus the Vacuum Cleaner problem was solved using a simple reflex agent; both rooms were cleaned in 3 actions and the program executed successfully.

Experiment No. 7: Breadth-First Search (BFS)

AIM

To write a Python program to implement Breadth-First Search, which traverses a graph level by level, and to find the shortest path (fewest edges) from a start node to a goal node.

ALGORITHM

1. Represent the graph as an adjacency list. Choose the start node A and the goal node G.
2. Create an empty queue (FIFO), insert the start node, and record it as visited with no parent.
3. Remove the node at the front of the queue and visit it. If it is the goal node, stop.
4. Insert every unvisited neighbour of the node at the rear of the queue, and remember the node as the parent of each neighbour.
5. Repeat steps 3 and 4 until the goal is found or the queue becomes empty.
6. Trace the parent links from the goal back to the start to obtain the shortest path; print the traversal order and the path.

PROGRAM

```
# Breadth-First Search on an undirected graph
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F", "G"],
    "F": ["C", "E"],
    "G": ["E"],
}

def bfs(start, goal):
    visited = [start]
    queue = deque([start])
    parent = {start: None}
    while queue:
        node = queue.popleft()                  # FIFO: oldest node first
        print(f"Visit {node:<3} queue after expansion:", end=" ")
        for nb in graph[node]:
            if nb not in parent:
                parent[nb] = node
                visited.append(nb)
                queue.append(nb)
        print(list(queue))
        if node == goal:
            break
    path, n = [], goal
    while n is not None:
        path.append(n)
        n = parent[n]
    return visited, path[::-1]

print("Adjacency list:")
for k, v in graph.items():
    print(f"  {k} -> {v}")
print()
visited, path = bfs("A", "G")
print("\nBFS traversal order:", " -> ".join(visited))
print("Shortest path A to G:", " -> ".join(path), f"({len(path) - 1} edges)")
```

OUTPUT

Terminal

```
$ python3 bfs.py
Adjacency list:
  A -> ['B', 'C']
  B -> ['A', 'D', 'E']
  C -> ['A', 'F']
  D -> ['B']
  E -> ['B', 'F', 'G']
  F -> ['C', 'E']
  G -> ['E']

Visit A   queue after expansion: ['B', 'C']
Visit B   queue after expansion: ['C', 'D', 'E']
Visit C   queue after expansion: ['D', 'E', 'F']
Visit D   queue after expansion: ['E', 'F']
Visit E   queue after expansion: ['F', 'G']
Visit F   queue after expansion: ['G']
Visit G   queue after expansion: []

BFS traversal order: A -> B -> C -> D -> E -> F -> G
Shortest path A to G: A -> B -> E -> G (3 edges)
$
```

RESULT

Thus Breadth-First Search was implemented; the traversal order and the shortest path A -> B -> E -> G were obtained and the program executed successfully.

Experiment No. 8: Depth-First Search (DFS)

AIM

To write a Python program to implement Depth-First Search, which explores a graph as deep as possible along each branch before backtracking, using a stack.

ALGORITHM

1. Represent the graph as an adjacency list. Choose the start node A and the goal node G.
2. Create an empty stack (LIFO) and push the start node together with the path from the start.
3. Pop the node on top of the stack. If it has already been visited, ignore it; otherwise mark it visited. If it is the goal node, stop.
4. Push the unvisited neighbours of the node onto the stack (in reverse order, so that the alphabetically first neighbour is explored first).
5. Repeat steps 3 and 4 until the goal is found or the stack becomes empty (backtracking happens automatically when a branch has no new nodes).
6. Print the order in which the nodes were visited and the path from the start to the goal.

PROGRAM

```
# Depth-First Search on an undirected graph (stack based)
graph = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F", "G"],
    "F": ["C", "E"],
    "G": ["E"],
}

def dfs(start, goal):
    stack = [(start, [start])]                  # (node, path from start)
    visited = []
    while stack:
        node, path = stack.pop()                # LIFO: newest node first
        if node in visited:
            continue
        visited.append(node)
        if node == goal:
            print(f"Visit {node:<3} goal reached")
            return visited, path
        for nb in reversed(graph[node]):        # reversed so that A-Z order is explored first
            if nb not in visited:
                stack.append((nb, path + [nb]))
        print(f"Visit {node:<3} stack after expansion (top on the right):", [n for n, _ in stack])
    return visited, None

print("Adjacency list:")
for k, v in graph.items():
    print(f"  {k} -> {v}")
print()
visited, path = dfs("A", "G")
print("\nDFS traversal order:", " -> ".join(visited))
print("Path found A to G  :", " -> ".join(path), f"({len(path) - 1} edges)")
```

OUTPUT

Terminal

```
$ python3 dfs.py
Adjacency list:
  A -> ['B', 'C']
  B -> ['A', 'D', 'E']
  C -> ['A', 'F']
  D -> ['B']
  E -> ['B', 'F', 'G']
  F -> ['C', 'E']
  G -> ['E']

Visit A   stack after expansion (top on the right): ['C', 'B']
Visit B   stack after expansion (top on the right): ['C', 'E', 'D']
Visit D   stack after expansion (top on the right): ['C', 'E']
Visit E   stack after expansion (top on the right): ['C', 'G', 'F']
Visit F   stack after expansion (top on the right): ['C', 'G', 'C']
Visit C   stack after expansion (top on the right): ['C', 'G']
Visit G   goal reached

DFS traversal order: A -> B -> D -> E -> F -> C -> G
Path found A to G  : A -> B -> E -> G (3 edges)
$
```

RESULT

Thus Depth-First Search was implemented; the traversal order A -> B -> D -> E -> F -> C -> G was obtained and the program executed successfully.

Experiment No. 9: Travelling Salesman Problem

AIM

To write a Python program to solve the Travelling Salesman Problem for five cities by the brute-force method, i.e. to find the shortest round trip that visits every city exactly once and returns to the starting city.

ALGORITHM

1. Represent the cities as A to E and store the distance between every pair of cities in a distance matrix. Fix city A as the starting city.
2. Generate every permutation of the remaining cities (4! = 24 orders); each permutation, preceded and followed by city A, is one tour.
3. For each tour, add up the distances of consecutive cities, including the return leg to the starting city, to get the tour cost.
4. Compare the cost with the best cost found so far (initially infinity); if it is smaller, store the tour as the best tour.
5. Repeat steps 3 and 4 for all the tours.
6. Print all the tours with their costs, and the shortest tour with its minimum cost.

PROGRAM

```
# Travelling Salesman Problem solved by brute force (all permutations)
from itertools import permutations

cities = ["A", "B", "C", "D", "E"]
dist = [
    [0, 12, 10, 19, 8],
    [12, 0, 3, 7, 6],
    [10, 3, 0, 2, 20],
    [19, 7, 2, 0, 4],
    [8, 6, 20, 4, 0],
]

def tour_cost(order):
    """Cost of the round trip start -> ... -> start."""
    cost = 0
    for i in range(len(order)):
        cost += dist[order[i]][order[(i + 1) % len(order)]]
    return cost

print("Distance matrix:")
print("    " + "".join(f"{c:>4}" for c in cities))
for i, row in enumerate(dist):
    print(f"{cities[i]:>3} " + "".join(f"{d:>4}" for d in row))
print()

best_cost, best_tour, count = float("inf"), None, 0
print(f"{'No.':<5}{'Tour':<28}Cost")
for perm in permutations(range(1, len(cities))):        # city A fixed as the start
    order = (0,) + perm
    cost = tour_cost(order)
    count += 1
    route = " -> ".join(cities[i] for i in order) + " -> " + cities[0]
    print(f"{count:<5}{route:<28}{cost}")
    if cost < best_cost:
        best_cost, best_tour = cost, route
print(f"\nTours examined = {count}")
print("Shortest tour  =", best_tour)
print("Minimum cost   =", best_cost)
```

OUTPUT

Terminal

```
$ python3 tsp.py
Distance matrix:
       A   B   C   D   E
  A    0  12  10  19   8
  B   12   0   3   7   6
  C   10   3   0   2  20
  D   19   7   2   0   4
  E    8   6  20   4   0

No.  Tour                          Cost
1    A -> B -> C -> D -> E -> A  29
2    A -> B -> C -> E -> D -> A  58
3    A -> B -> D -> C -> E -> A  49
4    A -> B -> D -> E -> C -> A  53
5    A -> B -> E -> C -> D -> A  59
6    A -> B -> E -> D -> C -> A  34
7    A -> C -> B -> D -> E -> A  32
8    A -> C -> B -> E -> D -> A  42
9    A -> C -> D -> B -> E -> A  33
10   A -> C -> D -> E -> B -> A  34
11   A -> C -> E -> B -> D -> A  62
12   A -> C -> E -> D -> B -> A  53
13   A -> D -> B -> C -> E -> A  57
14   A -> D -> B -> E -> C -> A  62
15   A -> D -> C -> B -> E -> A  38
16   A -> D -> C -> E -> B -> A  59
17   A -> D -> E -> B -> C -> A  42
18   A -> D -> E -> C -> B -> A  58
19   A -> E -> B -> C -> D -> A  38
20   A -> E -> B -> D -> C -> A  33
21   A -> E -> C -> B -> D -> A  57
22   A -> E -> C -> D -> B -> A  49
23   A -> E -> D -> B -> C -> A  32
24   A -> E -> D -> C -> B -> A  29

Tours examined = 24
Shortest tour  = A -> B -> C -> D -> E -> A
Minimum cost   = 29
$
```

RESULT

Thus the Travelling Salesman Problem was solved by brute force; the shortest tour A -> B -> C -> D -> E -> A with minimum cost 29 was found and the program executed successfully.

Experiment No. 10: A* Algorithm

AIM

To write a Python program to implement the A* search algorithm to find the least-cost path from a start cell to a goal cell in a grid containing obstacles, using the Manhattan-distance heuristic.

ALGORITHM

1. Represent the maze as a grid in which S is the start, G is the goal and # is a wall. Moves are allowed Up, Down, Left and Right with a cost of 1 each.
2. Define f(n) = g(n) + h(n), where g is the cost from the start to n and h is the Manhattan distance from n to the goal.
3. Put the start cell in the open list (priority queue) with g = 0.
4. Remove the cell with the smallest f from the open list. If it is the goal, trace the parent links back to the start to get the path and stop.
5. Otherwise generate the neighbouring cells that are inside the grid and are not walls; each has g + 1.
6. If a neighbour is new, or has been reached with a smaller g than before, record its parent and g and insert it into the open list with its f value.
7. Repeat steps 4 to 6 until the goal is found; print the path, its cost and the number of nodes expanded.

PROGRAM

```
# A* search algorithm on a grid (Manhattan-distance heuristic)
import heapq

GRID = [
    "S . . # . . .",
    ". # . # . # .",
    ". # . . . # .",
    ". # # # . # .",
    ". . . . . . G",
]
GRID = [row.split() for row in GRID]
ROWS, COLS = len(GRID), len(GRID[0])
for r in range(ROWS):
    for c in range(COLS):
        if GRID[r][c] == "S": START = (r, c)
        if GRID[r][c] == "G": GOAL = (r, c)

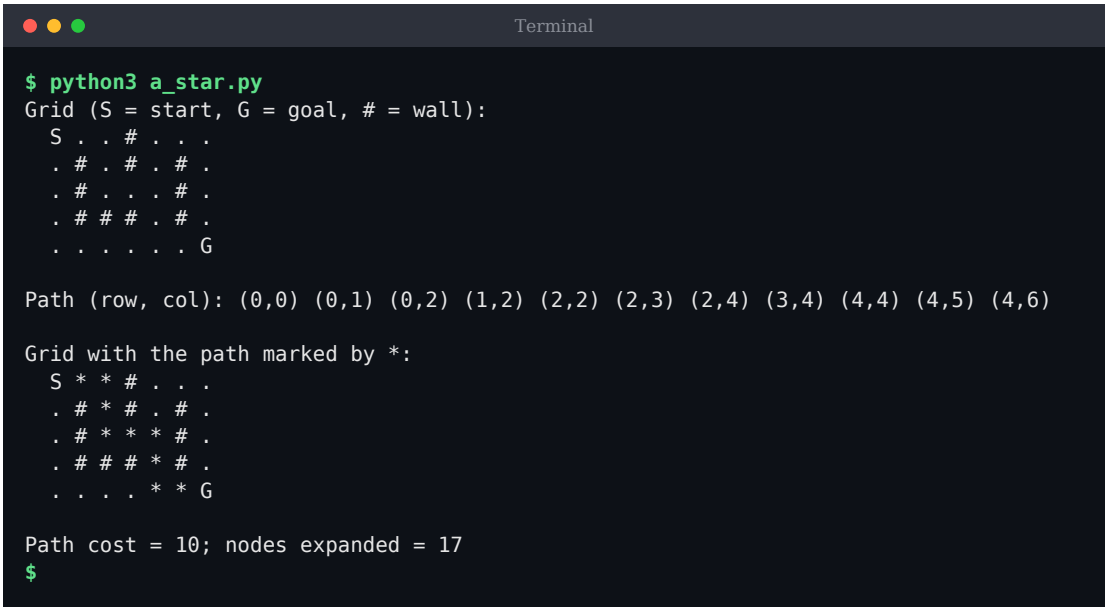
def h(p):
    """Manhattan distance to the goal (admissible heuristic)."""
    return abs(p[0] - GOAL[0]) + abs(p[1] - GOAL[1])

def astar():
    open_list = [(h(START), 0, START)]          # (f, g, cell)
    parent, best_g, expanded = {START: None}, {START: 0}, 0
    while open_list:
        f, g, cell = heapq.heappop(open_list)
        if cell == GOAL:
            path = []
            while cell is not None:
                path.append(cell)
                cell = parent[cell]
            return path[::-1], g, expanded
        if g > best_g[cell]:
            continue
        expanded += 1
        r, c = cell
        for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < ROWS and 0 <= nc < COLS and GRID[nr][nc] != "#":
                ng = g + 1
```

```
                if (nr, nc) not in best_g or ng < best_g[(nr, nc)]:
                    best_g[(nr, nc)] = ng
                    parent[(nr, nc)] = cell
                    heapq.heappush(open_list, (ng + h((nr, nc)), ng, (nr, nc)))
    return None, None, expanded

print("Grid (S = start, G = goal, # = wall):")
for row in GRID:
    print("  " + " ".join(row))
path, cost, expanded = astar()
print("\nPath (row, col):", " ".join(f"({r},{c})" for r, c in path))
print("\nGrid with the path marked by *:")
for r in range(ROWS):
    print("  " + " ".join("*" if (r, c) in path and GRID[r][c] == "." else GRID[r][c] for c in range(COLS)))
print(f"\nPath cost = {cost}; nodes expanded = {expanded}")
```

OUTPUT



RESULT

Thus the A* algorithm was implemented; the least-cost path of cost 10 was found and the program executed successfully.